CPE 439 Lecture 04

Review

All About Task

Delay Functions

Determinism

- Refers to the consistency of timing behavior in a system: ensures no timing constraints are violated
  - A deterministic system responds within predictable intervals regardless of system load
  - Roughly refers to reliability of system

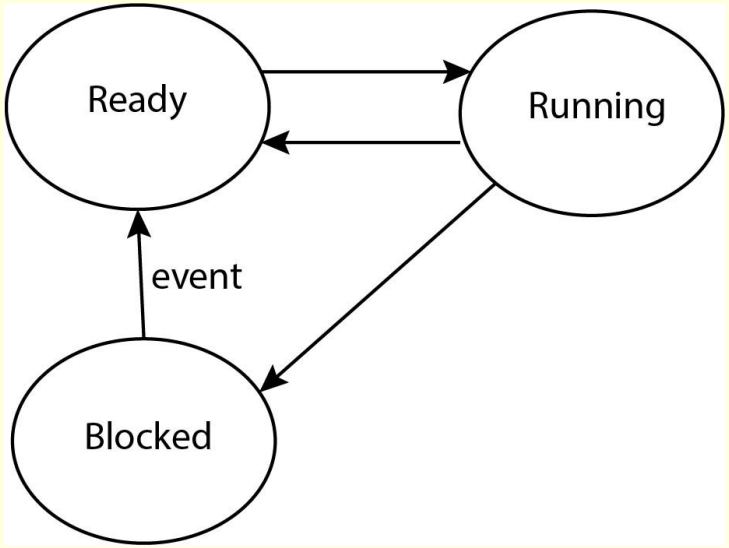
- How to help achieve determinism?
  - Minimizing unbounded delays
  - Use proper scheduling
  - Keeping ISRs short
  - Carefully managing shared resources

The RTOS (in General)

- System is divided into “Tasks”
  - Task: discrete processing unit (code) responsible for implementing task needs
  - Tasks are mini-programs
- Tasks are officially “created”
  - Task creation is a RTOS API call
- Scheduler is started
  - Scheduler takes control and never stops
  - Scheduler is code that executes, but is separate from code in Tasks

Task States

- Tasks can exist in three states
  - Running: task receiving processing time
  - Ready: task has something to do but waiting for higher priority task to complete
  - Blocked: task has nothing to do



- Tasks are not always running
- Task are designed to "block themselves"
- Scheduler responsible for all state changes

Time in an RTOS

- RTOSes: they have their own notion of time
  - RTOS time: subdivided into “ticks”

- Tick Implementation:
  - A timer that generates a periodic interrupt
  - Creates:
    - a periodic opportunity to “do something”
    - “time slicing” in the RTOS

The RTOS: the Idle Task

- The MCU always needs to be doing something
  - All created tasks can be blocked
  - It’s an MCU, something needs to be running

- The scheduler creates an idle task
  - Idle task “runs” but does nothing
  - The idle task has a lowest possible priority
    - Allows any other task to always run
    - The idle task runs when no other task is ready to run

The RTOS Mechanism

- The Scheduler
  - It's code that needs to run
    - If the scheduler is running tasks are not
    - If tasks are running the scheduler is not

- Scheduler "gets to execute" at two instances
  - the only times a context switch can be made

  1) At each tick interrupt
  2) When tasks choose to "block"

RTOS Pseudocode Example

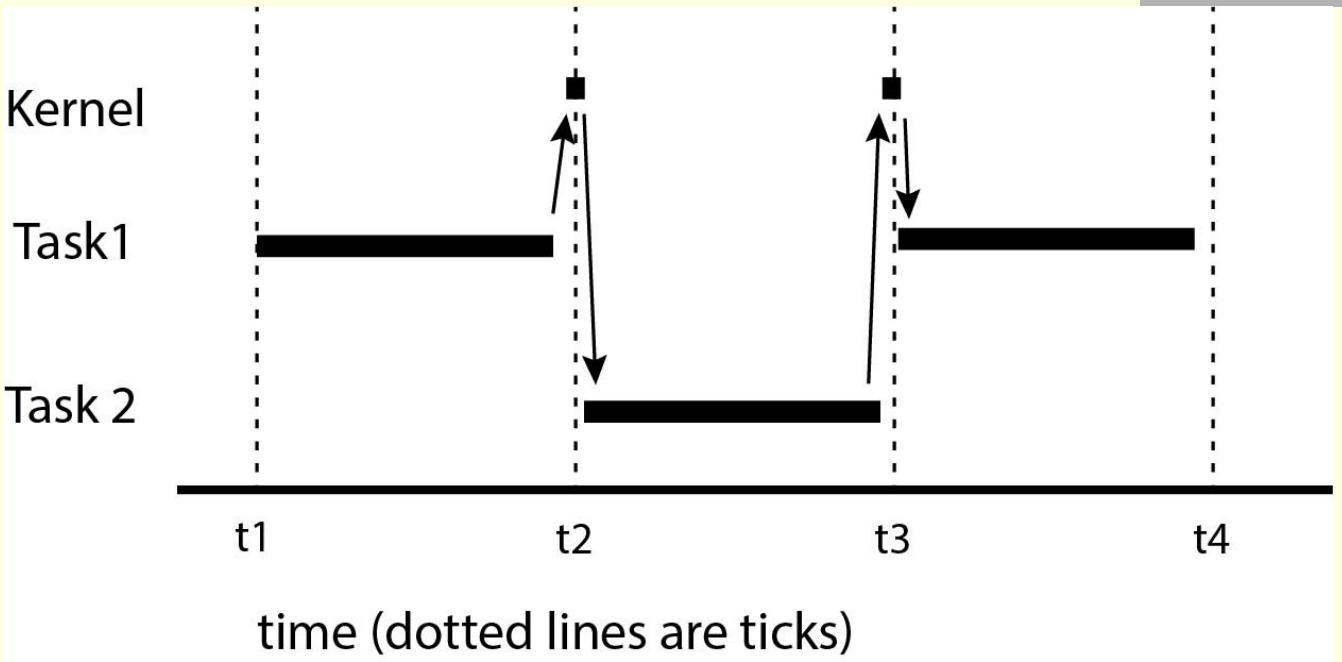
```
void Task1 (parameters)          // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

void Task2 (parameters)         // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=2); // two tasks same priority
    Create_task(Task2, priority=2);

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Scheduler (part of kernel) runs after every tick
  - Tasks have equal priorities:
  - Scheduler implements round robin scheduling to ensure tasks of equal priority run
  - When the kernel is running, the tasks are not

Tasks and Events

- Tasks have no need to always run
  - They're expected to block

- A block task: typically waits for one of two types of events; when event occurs, task is moved to ready state
  - <u>Temporal</u>: time related
    - Expiration of a delay
  - <u>Synchronization</u>
    - From an interrupt or other from other task's actions
    - Examples: Queues, semaphores, mutexes

RTOS Pseudocode Example

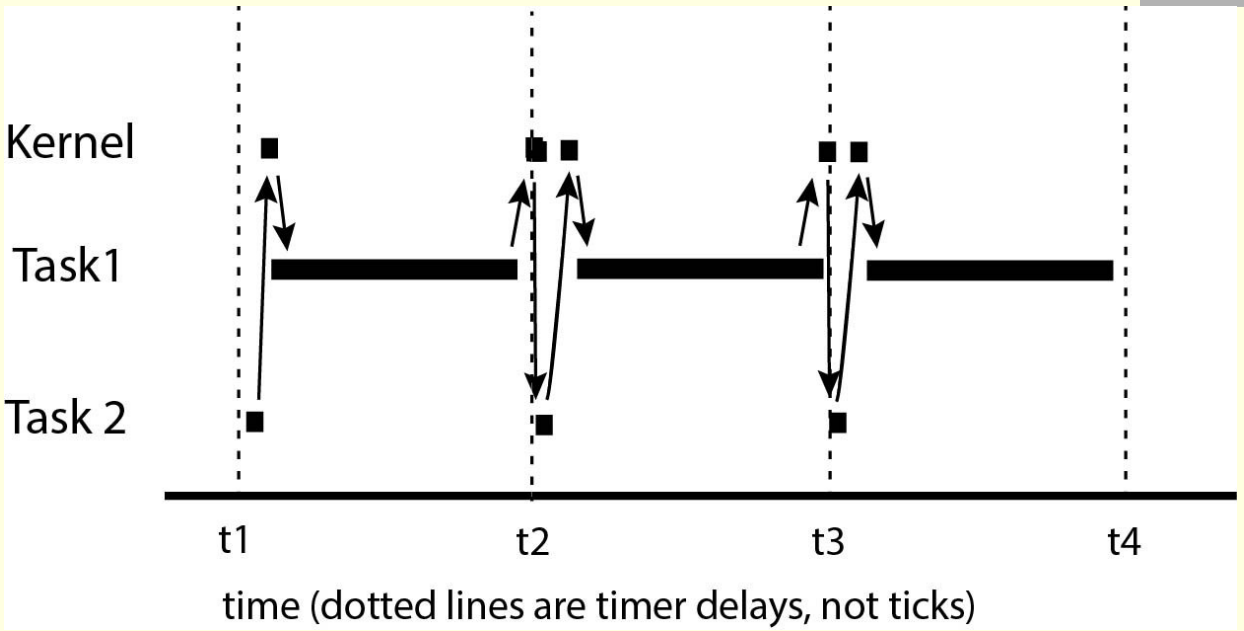
```
void Task1 (parameters)          // continuous processing task
{
   for (;;) {
                                 // code defining task
   }
}

void Task2 (parameters)
{
   for (;;) {
       Delay_API(#_ticks)          // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=1); // lower priority task
    Create_task(Task2, priority=2);

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Timing is not in ticks
  - When Task2 unblocks, it “pre-empts” Task1; a context switch happens to transfers Task2 to running and Task1 to ready
  - Scheduler runs when Task2 blocks (call delay function) and unblocks (timer delay expires)

Leaving the Blocked State

- Blocked state only goes to ready state
  - Scheduler decides which states will be run based on current task running and the state of those tasks

- Tasks can leave blocked state in two ways
  - The awaited event occurs
  - The event doesn't occur but task "times-out"
    - Times out: a bounded delay… strive to not have unbounded delays

Multiple Instances of a Task

- There can be multiple tasks created based on the same Task definition
  - All the tasks reference the same task code
    - Multiple tasks can run the same code
    - But not at the same time

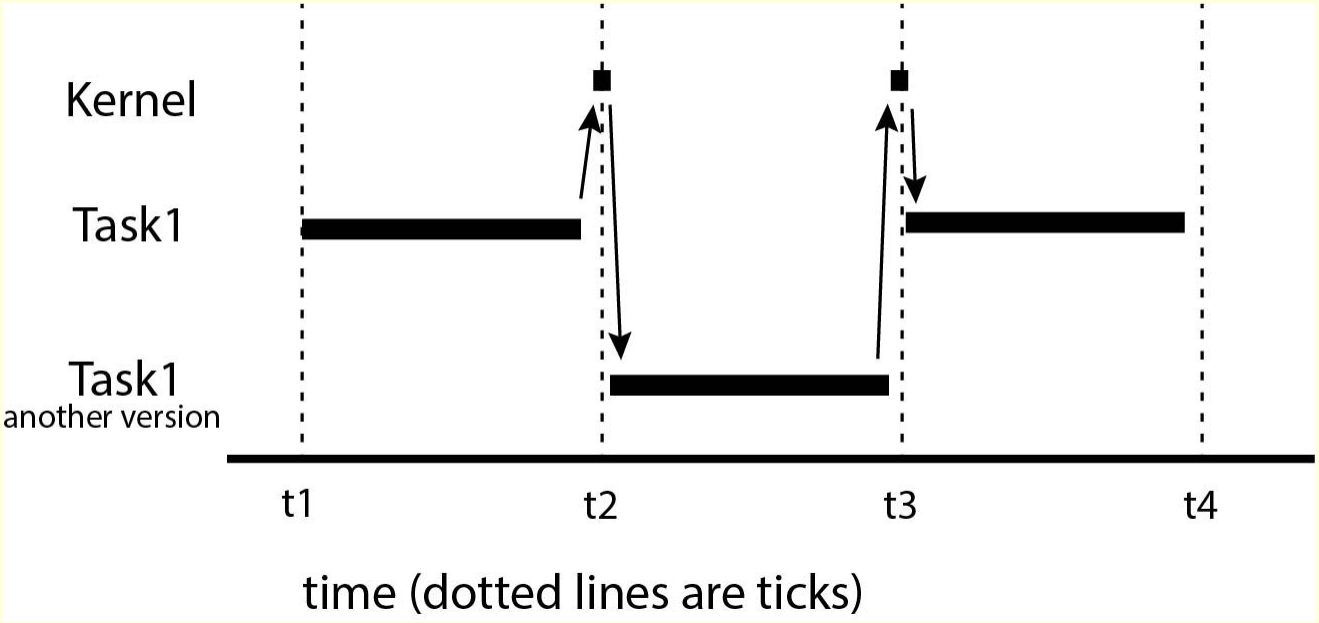
RTOS Pseudocode Example

```
void Task1 (parameters)          // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=2); // two tasks same priority
    Create_task(Task1, priority=2); // another version of Task1

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Scheduler (part of kernel) runs after every tick
  - Tasks have equal priorities:
  - Scheduler implements round robin scheduling to ensure tasks of equal priority run
  - When the kernel is running, the tasks are not

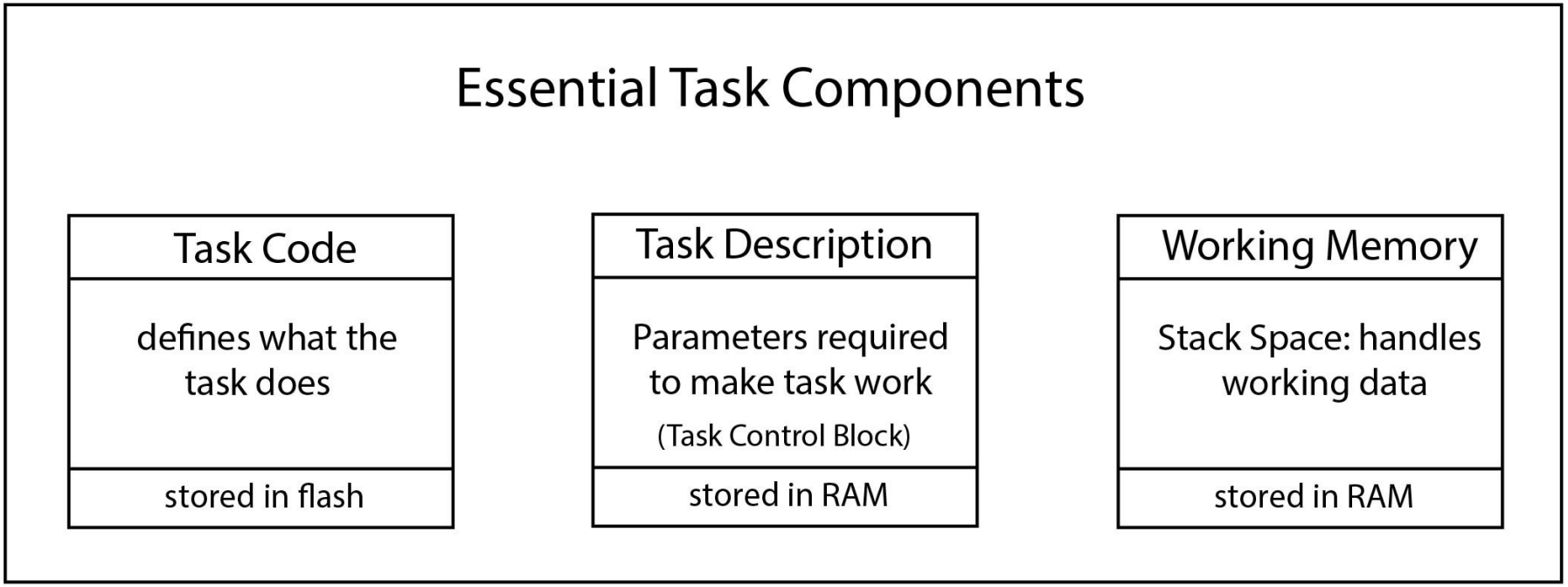
Task Details

- Defined by C function
  - Task code never “returns”
    - No return statement
    - Return type: void
- Acts like a program all its own
  - Tasks contain all the information to make it seem like an independent program

```
void Task1 (void* parameters)        // task definition
{
   for (;;) {
      // code defining task
   }
}
```

Task Guts

- What it takes to make Task “functional”



Task Description

- All pertinent information to make Task function as a “mini program”
- Task Control Block (TCB)
  - Contains information that makes task work
  - TCB contents are RTOS-dependent
    - FreeRTOS TCB???
- Minimally each task has its own
  - Copy of MCU registers
  - Copy of Program Counter
  - Temporary storage: stack

RTOS Context Switch

- A running task swaps with a ready task
  - Ready task has higher priority
- <u>Context switch</u>
  - Running task saves registers & PC in TCB
  - Ready task loads it registers into MCU's registers
  - Rady task loads it PC in the MCU's PC

- RTOSes are typically judged by how fast they can make context switches
  - Overhead: no meaningful work is done

Task Creation

- Tasks are created by API call
  - Function prototype

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        const configSTACK_DEPTH_TYPE uxStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask
                      );
```

Task Creation

```
retval xTaskCreate( Task code,
                    Task name,
                    Stack size,
                    Task parameter,
                    Priority,
                    Task Handle                    );
```

Arguments:

Task code: name of task defining function
Task name: unused name of task
Stack size: depth of stack in words
Task parameter: a value that can be sent to stack
Priority: task priority
Task Handle: reference to a task

Return Value:

retval: Boolean verifying task creation

Task Deletion

- API function to delete task
  - Code that deletes a task must know the handle of the task being deleted
  - Other RTOS APIs need task handle in some cases

```
void vTaskDelete( TaskHandle_t xTask );

Arguments: the handle to the task being deleted
Return value: none

void vTaskDelete( NULL); // deletes task where called from
```

Static Variable Declarations

- Static defines the storage class
  - Variable declared static go in flash memory
  - Thus, they're permanent
- Automatic is a storage class
  - All variable definitions not static default to automatic
  - Automatic variable declarations go on stack
  - Stacks are defined in RAM; data is not permanent

Static Variable Declared in Task Definition

- my_var1: on copy shared by all instances of the Task1
- my_var2: exists on task's stack
  - Each task instance has own copy

```
void Task1 (parameters)     // continuous processing task
{
   static int my_var1;     // static storage class
   int my_var2;            // automatic storage class

   for (;;) {
      // code defining task
   }
}

int main (void)
{
    Create_task(Task1, priority=2); // two tasks same priority
    Create_task(Task1, priority=2); // another version of Task1

    Start_scheduler();

    for (;;) ;
   }
}
```

RTOS Delay APIs

- Task in running state can call a delay API
  - Causes the task to block for a given amount of time
  - Delay times are measured in ticks
- Two versions of delay APIs

```
void vTaskDelay( const TickType_t xTicksToDelay );

void vTaskDelayUntil( TickType_t *pxPreviousWakeTime,
                      const TickType_t xTimeIncrement );
```

vTaskDelay

- Call to API causes task to block for a given amount of time
  - Block time amount: number of ticks starting from the tick count when vTaskDelay is called

```
//void vTaskDelay( const TickType_t xTicksToDelay );

void Task( void *Parameters )
{
    // constant defined by system for conversion
    TickType_t xDelay = 500 / portTICK_PERIOD_MS;

    for( ;; )
    {
       // do something meaningful
       vTaskDelay( xDelay );  // block 500ms
    }
}
```

vTaskDelayUntil

- Call to API causes task to block for a given amount of time
  - Block time amount: an absolute time when the task unblocks
    - Measured in a “chunk of time”
    - Chunk of time measured in ticks

```
void vTaskDelayUntil( TickType_t *pxPreviousWakeTime,
                      const TickType_t xTimeIncrement );

Arguments:

      PreviousWakeTime:  (time base for delay)
      TimeIncrement: cycle time persion

Task unblocks at: TimeIncrement + PreviousWakeTime
```

vTaskDelayUntil

- Example usage:

```
void vTaskFunction( void * pvParameters )
{
    TickType_t xLastWakeTime;
    const TickType_t xFrequency = 10;  // ten ticks

    xLastWakeTime = xTaskGetTickCount(); // Init variable

    for( ;; )
    {
       // do something meaningful
       vTaskDelayUntil( &xLastWakeTime, xFrequency );
    }
}
```

Differences: `vTaskDelay & vTaskDelayUntil`

- vTaskDelayUntil provides a periodic delay because it's based on the time relative to the last blocking call
  - And thus immune to "other stuff"

- vTaskDelay could have other stuff going on before API is called
- Before vTaskDelay call, a task could:
  - have other processing to do
  - A task could be pre-empted

Summary

- Tasks as “mini programs”; they store all the variables they need as part of the task in order to look like a mini program.
- Multiple versions of the same task can exist
- Two Delay API Calls
  - Task block upon delay call and unblocks later
    - vTaskDelay()
      - Measure time from API call into future
    - vTaskDelayUntil()
      - Measures time from previous API call to assigned time chunk

Daily Terms and Definitions

- Task Control Block (TCB): a data structure that the kernel uses to store all necessary information about a task, including the task's current state, priority, and resource ownership.

Experiment 1: Tasks and Blinking

- Purpose: get used to FreeRTOS tasks and doing something on the dev board
- Assignment: Write a program that:
  - Initially blinks the green LED approximately 30 times at a “viewable” rate (50% duty cycle).
  - Then blinks LED 30 times are a faster rate but still viewable (50% duty cycle).
  - Then LED then transitions to a blinking the LED in a heartbeat manner ten times, then stays forever on.

Experiment 1

- <u>Constraints</u>
  - Each of the LED blink patterns should be done with a separate task.
    - Don’t allow the IDE tools to create more than one task, which means you’ll need to write code to create tasks from an existing task
    - Task creation must happen after the scheduler has started
  - At least one of those blink rates need to use FreeRTOS or CMSIS delays rather than using dumb delays